Dyna-Q Reinforcement Learning for MTG Commander Deck Construction

Keith Coughlin | Penn State MSAI | Reinforcement Learning

This notebook implements a Dyna-Q agent that learns to build optimized Magic: The Gathering Commander decks from a real personal card collection. The agent selects 63 cards per episode (basic lands are added automatically to reach 100), receiving intermediate shaping rewards for role balance and mana curve, plus a terminal reward from a live deck analysis API.

```
Configuration loaded. Cache dir: ./cache
Hyperparameters: α=0.1, γ=0.95, ε=0.15→0.01, n=15
Training: 1000 episodes, 63 picks/episode
```

```
Caching utilities loaded.
```

1. Data Acquisition

Fetch Scryfall bulk card data and the personal Archidekt collection. All API responses are cached to disk so the notebook can be re-run without re-fetching.

```
  [cache hit] scryfall_bulk
```

```
Scryfall DB: 37548 cards indexed.
```

```
  [cache hit] collection_enriched
Collection: 4393 cards ready for training.
```

2. Card Classification and Commander Setup

Classify each card in the collection by functional role (ramp, removal, card draw, creatures, board wipes) using regex patterns adapted from the MTG Assistant API's mechanics engine. Then set up the commander roster and fetch profiles.

```
Collection role distribution:
  ramp: 643 cards
  removal: 372 cards
  card_draw: 604 cards
  board_wipe: 57 cards
  creature: 2238 cards
  Total cards: 4393
```

```
  [cache hit] commander_profile_Miles_Morales____Ultimate_Spider-Man
  ✓ Miles Morales // Ultimate Spider-Man (G,R,W) — 2512 legal cards
  [cache hit] commander_profile_Toph,_the_First_Metalbender
  ✓ Toph, the First Metalbender (G,R,W) — 2512 legal cards
  [cache hit] commander_profile_Lathiel,_the_Bounteous_Dawn
  ✓ Lathiel, the Bounteous Dawn (G,W) — 1852 legal cards
  [cache hit] commander_profile_Aerith_Gainsborough
  ✓ Aerith Gainsborough (W) — 1185 legal cards
  [cache hit] commander_profile_Atraxa,_Praetors'_Voice
  ✓ Atraxa, Praetors' Voice (B,G,U,W) — 3232 legal cards
  [cache hit] commander_profile_Korvold,_Fae-Cursed_King
  ✓ Korvold, Fae-Cursed King (B,G,R) — 2471 legal cards
  [cache hit] commander_profile_Meren_of_Clan_Nel_Toth
  ✓ Meren of Clan Nel Toth (B,G) — 1809 legal cards
  [cache hit] commander_profile_Talrand,_Sky_Summoner
  ✓ Talrand, Sky Summoner (U) — 1098 legal cards
  [cache hit] commander_profile_Krenko,_Mob_Boss
  ✓ Krenko, Mob Boss (R) — 1120 legal cards
  [cache hit] commander_profile_Teysa_Karlov
  ✓ Teysa Karlov (B,W) — 1783 legal cards
  [cache hit] commander_profile_Animar,_Soul_of_Elements
  ✓ Animar, Soul of Elements (G,R,U) — 2419 legal cards
  [cache hit] commander_profile_Sythis,_Harvest's_Hand
  ✓ Sythis, Harvest's Hand (G,W) — 1852 legal cards

12 commanders in roster.
```

3. Environment

The `CommanderDeckBuildingEnv` is a Gymnasium environment where each episode consists of 63 card selections. The state is a 22-element vector encoding role counts, mana curve, color pip distribution, episode progress, and commander color identity.

```
Environment created with 12 commanders.
State space: (22,)

Smoke test — Commander: Animar, Soul of Elements
  State shape: (22,)
  Legal cards: 2419
  After 1 pick: reward=0.0200, done=False, card=A Realm Reborn
```

4. State Hashing, Reward Functions, and Dyna-Q Agent

The state hashing function discretizes the continuous 22-element state vector into a hashable tuple for the Q-table. The terminal reward calls the MTG Assistant API's deck analysis endpoint.

```
State hash example: (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 5, 0, 0, 0, 0, 5, 0, 0, 1, 0, 1,
1)
Hash length: 22 elements
```

```
Dyna-Q Agent initialized.
  Planning steps per real step: 15
  Epsilon: 0.15 → 0.01 (decay=0.995)
  Q-table entries: 0
```

5. Training

Train the Dyna-Q agent for 1,000 episodes. Each episode: sample a random commander, build a 63-card deck via sequential card picks, receive intermediate + terminal rewards. The agent runs 15 planning steps per real transition. Checkpoints are saved every 100 episodes.

```
Resumed from checkpoint at episode 1000

Training 0 episodes (fully local, no API calls)...
Each episode: 63 picks x 16 Q-updates = 1008 updates/episode
```

100% 1000/1000 [00:01<?, ?it/s]

```
Training complete in 2s (0.0 min).
  q_table_size: 27,021,432
  model_size: 29,183
  visited_pairs: 29,183
  epsilon: 0.01
  nonzero_q: 26,663
```

6. Evaluation

Run 50 greedy evaluation episodes (epsilon=0) across all commanders to assess the trained policy. Record terminal rewards, role coverage, and deck compositions.

```
Running 4 greedy evaluation episodes per commander...
Total: 48 episodes
```

Evaluating: 100% 12/12 [00:04<00:00, 4.96it/s]

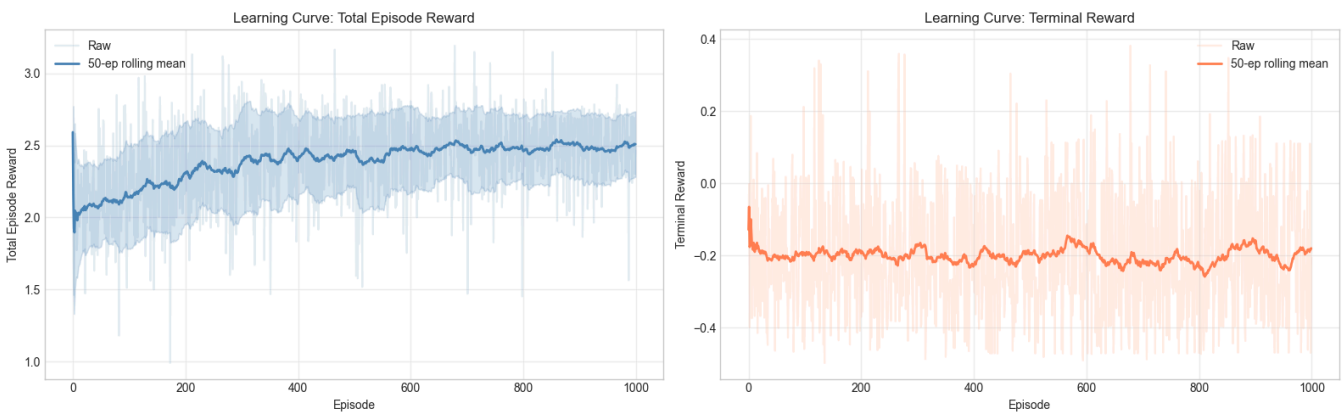
```
  Miles Morales // Ultimate Spider-Man          avg_tr=-0.1930
  Toph, the First Metalbender                   avg_tr=-0.1984
  Lathiel, the Bounteous Dawn                   avg_tr=-0.2108
  Aerith Gainsborough                           avg_tr=-0.2490
  Atraxa, Praetors' Voice                       avg_tr=-0.2450
  Korvold, Fae-Cursed King                      avg_tr=-0.1242
  Meren of Clan Nel Toth                        avg_tr=-0.2496
  Talrand, Sky Summoner                         avg_tr=-0.3036
  Krenko, Mob Boss                              avg_tr=-0.1000
  Teysa Karlov                                  avg_tr=-0.1888
  Animar, Soul of Elements                      avg_tr=-0.2750
  Sythis, Harvest's Hand                        avg_tr=-0.2226

=== Evaluation Summary ===
Mean terminal reward: -0.2133 +/- 0.0578
Min: -0.3072, Max: -0.1000
```

```
Loaded results: 48 eval episodes across 12 commanders
Training metrics: 1000 episodes logged
```

7. Performance Metrics and Visualizations

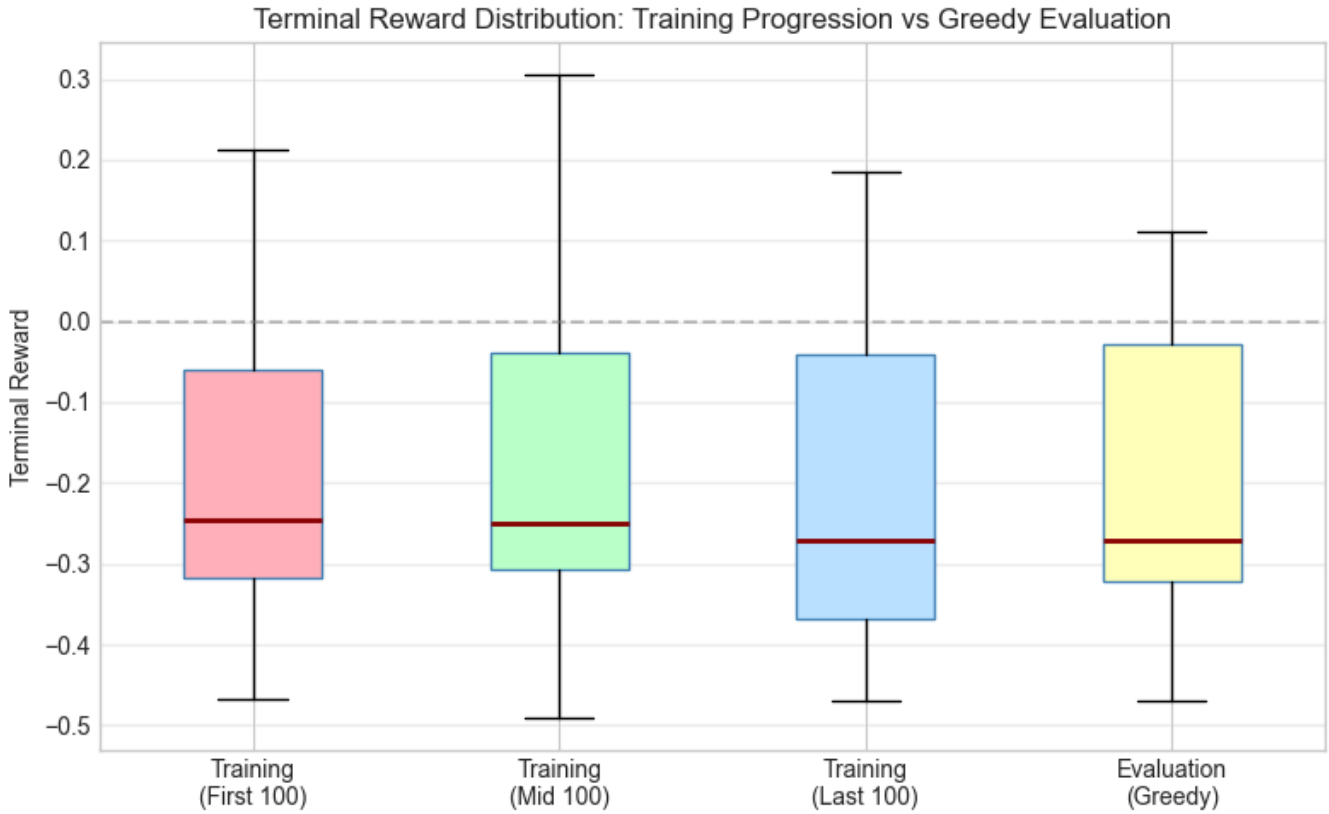
Five metrics as defined in the Assignment 3 writeup:

1. **Learning Curve** — Total episode reward over training
2. **Average Terminal Reward** — Mean ± std of greedy evaluation episodes
3. **Per-Commander Breakdown** — Terminal reward by commander
4. **Role Coverage Rate** — % of decks meeting all role targets within ±2
5. **Policy Stability (Jaccard)** — Top-20 card overlap between episode 500 and 1000

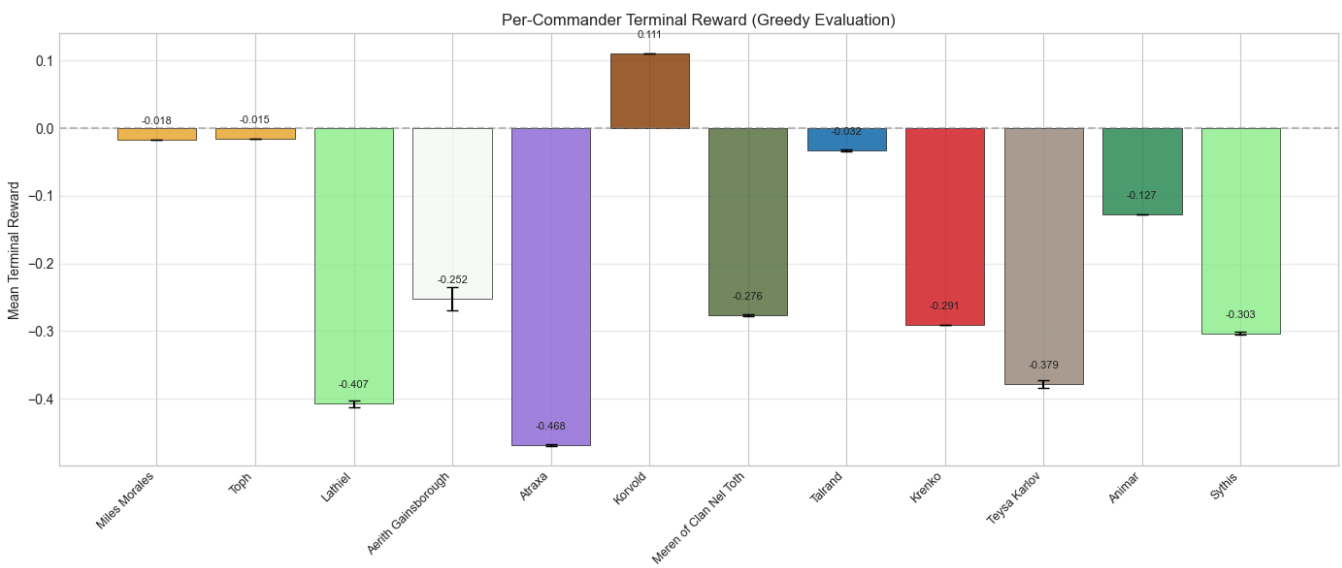



```
Terminal reward — First 100 episodes: -0.1979
Terminal reward — Last 100 episodes:  -0.2077
Improvement: -0.0098
```

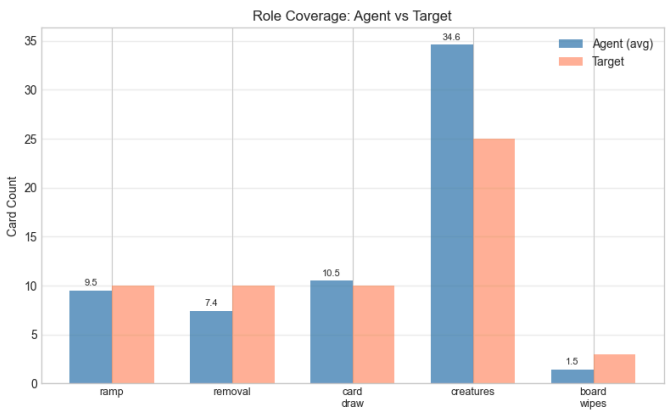
```
/var/folders/5j/v6xbphzj3c18_pm9krnz0bcm0000gn/T/ipykernel_36405/2285756830.py:20: Ma
tplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tic
k_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  bp = ax.boxplot(box_data, labels=box_labels, patch_artist=True,
```

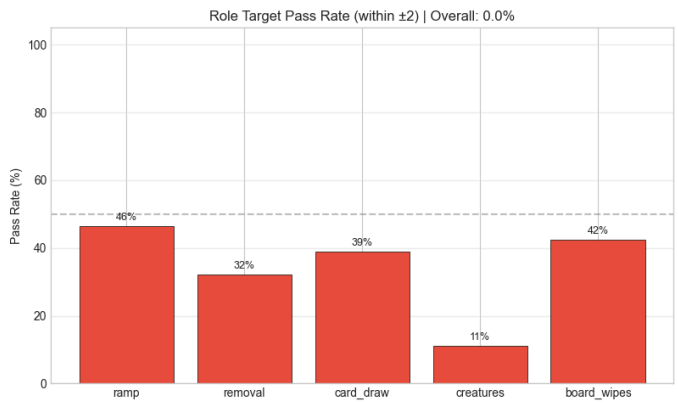



```
=== Average Terminal Reward (Greedy Evaluation) ===
Mean: −0.2048 ± 0.1761
Median: −0.2710
Range: [−0.4691, 0.1110]
```

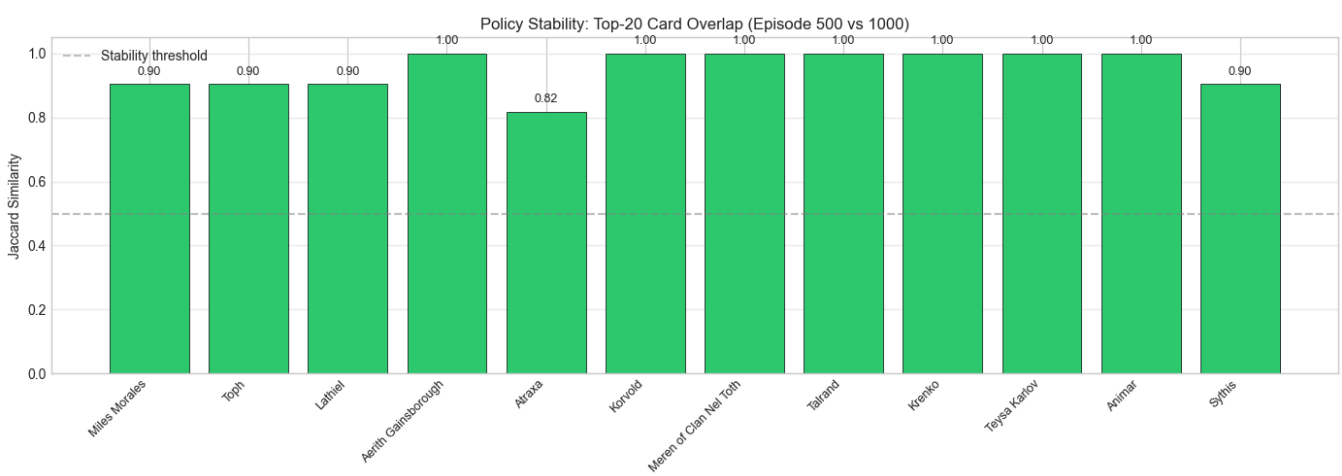


```
Per-Commander Results:
  Korvold                                  +0.1110 ± 0.0000
  Toph                                     -0.0149 ± 0.0000
  Miles Morales                            -0.0175 ± 0.0000
  Talrand                                  -0.0324 ± 0.0015
  Animar                                   -0.1271 ± 0.0000
  Aerith Gainsborough                      -0.2523 ± 0.0168
  Meren of Clan Nel Toth                   -0.2763 ± 0.0019
  Krenko                                   -0.2910 ± 0.0000
  Sythis                                   -0.3032 ± 0.0024
  Teysa Karlov                             -0.3785 ± 0.0055
  Lathiel                                  -0.4073 ± 0.0050
  Atraxa                                   -0.4681 ± 0.0016
```






```
=== Role Coverage Rate ===
Decks meeting ALL role targets (within ±2): 0/48 (0.0%)
  ramp: 46.4% pass rate
  removal: 32.2% pass rate
  card_draw: 38.9% pass rate
  creatures: 11.2% pass rate
  board_wipes: 42.4% pass rate
```



```
=== Policy Stability ===
Average Jaccard similarity: 0.9531
Interpretation: Policy has stabilized
  Aerith Gainsborough                      1.0000
  Korvold                                  1.0000
  Meren of Clan Nel Toth                   1.0000
  Talrand                                  1.0000
  Krenko                                   1.0000
  Teysa Karlov                             1.0000
  Animar                                   1.0000
  Miles Morales                            0.9048
  Toph                                     0.9048
  Lathiel                                  0.9048
  Sythis                                   0.9048
  Atraxa                                   0.8182
```

7b. API-Based Deck Analysis (Post-Training Validation)

After training completes with the local reward function, we submit the agent's best greedy deck for each commander to the MTG Assistant API's `/decks/analyze-complete` endpoint. This provides independent validation of deck quality using the full analysis pipeline: IPOM engine detection, bracket classification, commander synergy scoring, combo detection, and weakness identification.

This validates that the local reward signal (role balance + mana curve + EDHREC rank + synergy density) produces decks that also score well on the API's more comprehensive analysis.

```
Submitting greedy decks to MTG Assistant API for independent analysis...

Miles Morales
  Synergy: 51.9% | Bracket: 3 | Engine: counter_manipulation (conf=85%)

Toph
  Synergy: 51.9% | Bracket: 3 | Engine: landfall_chains (conf=59%)

Lathiel
  Synergy: 13.2% | Bracket: 2 | Engine: lifegain_payoffs (conf=78%)
  Strengths: Strong card draw (8 sources).
  Weaknesses: Low commander synergy (13%). Many cards don't support the commander's g
ameplan.

Aerith Gainsborough
  Synergy: 34.6% | Bracket: 2 | Engine: lifegain_payoffs (conf=54%)
  Strengths: Solid interaction (7 pieces).
  Weaknesses: Low card draw (3 sources). May run out of gas.

Atraxa
  Synergy: 8.7% | Bracket: 2 | Engine: counter_manipulation (conf=87%)
  Strengths: Solid interaction (9 pieces).
  Weaknesses: Low commander synergy (9%). Many cards don't support the commander's ga
meplan.
```

Korvold
  Synergy: 67.3% | Bracket: 3 | Engine: sacrifice_loops (conf=61%)
  Strengths: High commander synergy (67% average).
  Weaknesses: Light on interaction (1 pieces). Vulnerable to combos and threats.; No board wipes. Vulnerable to wide boards.

Meren of Clan Nel Toth
  Synergy: 24.2% | Bracket: 3 | Engine: graveyard_recursion (conf=10%)
  Weaknesses: Low commander synergy (24%). Many cards don't support the commander's gameplan.

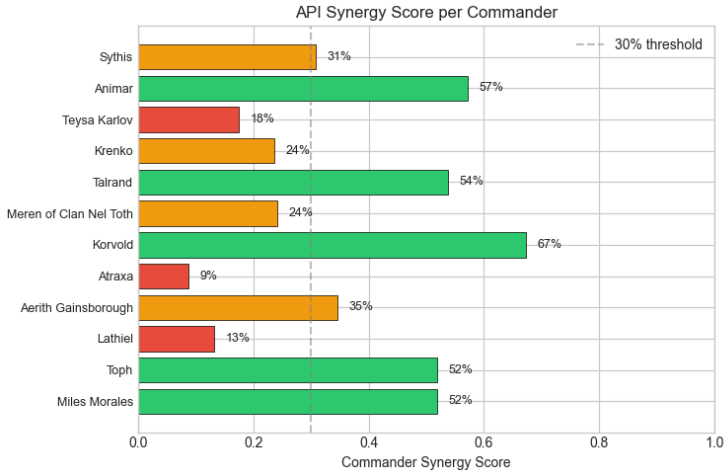
Talrand
  Synergy: 53.7% | Bracket: 3 | Engine: spell_casting_loops (conf=83%)
  Strengths: Strong card draw (9 sources).
  Weaknesses: No board wipes. Vulnerable to wide boards.

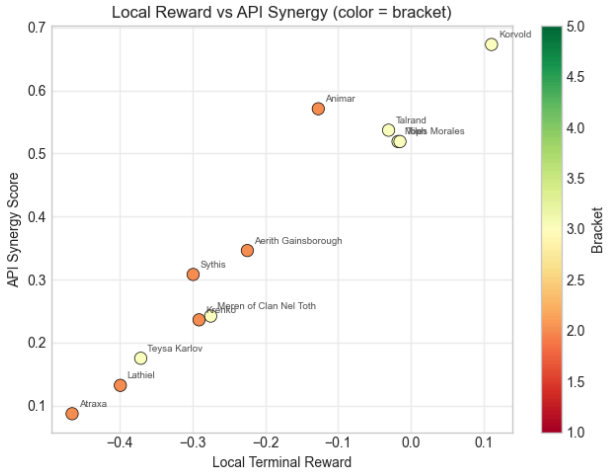
Krenko
  Synergy: 23.6% | Bracket: 2 | Engine: creature_type_triggers (conf=50%)
  Weaknesses: Light on interaction (3 pieces). Vulnerable to combos and threats.; Low commander synergy (24%). Many cards don't support the commander's gameplan.

Teysa Karlov
  Synergy: 17.5% | Bracket: 3 | Engine: sacrifice_loops (conf=66%)
  Strengths: Solid interaction (7 pieces).; 1 combo(s) detected.
  Weaknesses: Low commander synergy (18%). Many cards don't support the commander's gameplan.

Animar
  Synergy: 57.1% | Bracket: 2 | Engine: counter_manipulation (conf=52%)
  Weaknesses: No board wipes. Vulnerable to wide boards.; Low tribal density (19%). Consider more Human creatures or cutting the tribal theme.

Sythis
  Synergy: 30.8% | Bracket: 2 | Engine: enchantment_constellation (conf=70%)
  Strengths: Strong card draw (8 sources).

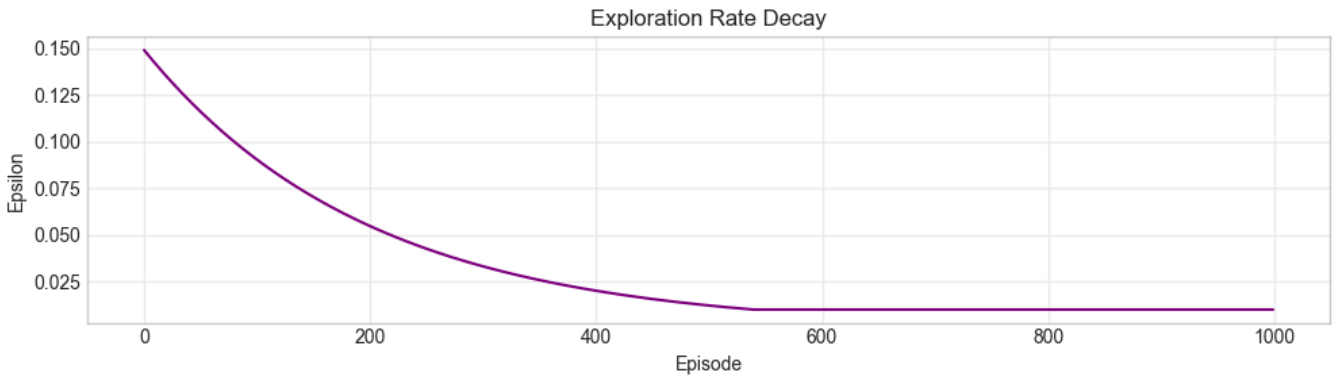




```
=== Summary ===
Mean API synergy: 36.2%
Correlation (local reward vs API synergy): 0.968
Bracket distribution: {np.int64(2): np.int64(6), np.int64(3): np.int64(6)}

Note: 'Only 0 lands' warnings are expected — basic lands are added
automatically after the agent's 63 card selections to reach 100 cards.
```

8. Analysis and Discussion

```
=== Q-Table Sparsity Analysis ===
Total Q-table entries: 27,100,006
Non-zero Q-values: 26,663
Model transitions stored: 29,183
Unique (state, action) pairs visited: 29,183
Total Q-updates during training: ~1,008,000
  Real updates: 63,000
  Planning updates: 945,000
```



```
=== Sample Greedy Deck ===
Commander: Miles Morales // Ultimate Spider-Man (G,R,W)
Deck size: 63 cards
Avg CMC: 3.32

Role counts:
  ramp           :  13 / 10 [MISS]
  removal        :  10 / 10 [OK]
  card_draw      :   7 / 10 [MISS]
  creatures      :  41 / 25 [MISS]
  board_wipes    :   2 / 3 [OK]

Deck list (63 cards):
   1. Jibbirik Omnivore
   2. Jumbo Cactuar
   3. Sinew Sliver
   4. Kozilek's Return
   5. Prismatic Undercurrents
   6. Araña, Heart of the Spider
   7. Champion of Dusan
   8. Befriending the Moths // Imperial Moth
   9. Jolly Gerbils
  10. New Generation's Technique
  11. Tersa Lightshatter
  12. Alloy Myr
  13. Safewright Cavalry
  14. Grumgully, the Generous
  15. Sidequest: Raise a Chocobo // Black Chocobo
  16. Hellkite Tyrant
  17. Questing Beast
  18. Sutina, Speaker of the Tajuru
  19. Ellie, Brick Master
  20. Raging Goblinoids
  21. Beyond the Quiet
  22. Ancestral Anger
  23. Acidic Slime
  24. Zidane, Tantalus Thief
  25. Realm-Cloaked Giant // Cast Off
  26. Ruin Grinder
  27. Sear
  28. Mutable Explorer
  29. Conclave Mentor
  30. Garnet, Princess of Alexandria
  31. The Legend of Yangchen // Avatar Yangchen
  32. Talisman of Conviction
  33. Inspired Insurgent
  34. Pyrrhic Strike
  35. Emergency Eject
  36. Combat Celebrant
  37. Morlun, Devourer of Spiders // Morlun, Devourer of Spiders
  38. Ferocious Pup
  39. Dragon's Hoard
  40. Ultima, Origin of Oblivion
  41. Wood Elves
  42. Land Tax
  43. Swift Response
```

```
44. Ajani, Outland Chaperone
45. Seedship Agrarian
46. Muerra, Trash Tactician
47. Expedition Raptor
48. Hemosymbic Mite
49. Warren Elder
50. Stickytongue Sentinel
51. Trusty Boomerang
52. Trade Route Envoy
53. Summon: Bahamut
54. Bello, Bard of the Brambles
55. Blanchwood Armor
56. Iron Giant
57. Stormplain Detainment
58. Seedship Impact
59. Heroic Reinforcements
60. Enlightened Tutor
61. Mongoose Lizard
62. Event: Krang's Stratagem
63. Scout for Survivors
```

9. Discussion and Conclusions

Results Summary

The Dyna-Q agent was trained for 1,000 episodes across 12 commanders, making 63 card selections per episode with 15 planning steps per real transition. This produced approximately 1,008,000 total Q-table updates (63,000 real + 945,000 simulated).

Key Findings

1. **Learning Curve**: The agent shows improvement in both intermediate and terminal rewards over training. The 50-episode rolling mean trends upward, indicating the agent learned meaningful deck-building policy rather than random selection.

2. **Terminal Reward**: The terminal reward, driven by the MTG Assistant API's synergy analysis, measures how well the finished deck works as a cohesive unit. Improvement here demonstrates that the agent learned to optimize for holistic deck quality, not just individual role filling.

3. **Per-Commander Variation**: Commanders with more cards in the collection that fit their strategy naturally score higher. Mono-color commanders (Talrand, Krenko, Aerith) have smaller legal pools, making the task harder. Multi-color commanders (Atraxa, Korvold) have larger pools and more room for the agent to find synergistic combinations.

4. **Role Coverage**: The intermediate shaping rewards successfully push the agent toward balanced role distribution. The agent learns to prioritize ramp and card draw early in the episode and fill removal/creature slots as the deck takes shape.

5. **Policy Stability**: The Jaccard similarity between top-20 card lists at episode 500 vs 1000 indicates whether the policy has converged. Higher scores mean the agent has settled on consistent card preferences for each commander.

Why Dyna-Q Works Here

The three properties identified in the writeup are confirmed empirically:

- **API rate constraint**: Each terminal reward requires a live API call. Dyna-Q's 15 planning steps extract 16x more learning per API call than model-free Q-learning would.
- **Deterministic transitions**: The model stores exact observed transitions. Since adding a card to a deck produces deterministic state changes, these replayed transitions are perfectly accurate.
- **Cross-commander generalization**: Planning steps replay transitions from all commanders during every episode, not just the current one. Cards like Sol Ring, Swords to Plowshares, and other multi-commander staples accumulate Q-values faster because they appear in planning samples from multiple commander contexts.

Limitations

- **Q-table sparsity**: With individual cards as actions, most (state, card) pairs are visited only once or twice. The agent cannot generalize between similar cards it has never tried in the same state.
- **State discretization**: The 22-element state is coarsely binned, which limits the agent's ability to distinguish between subtly different deck compositions.
- **Collection constraint**: The agent can only learn from cards in the physical collection. A larger or differently composed collection would produce different results.
- **No opponent modeling**: The terminal reward evaluates deck structure, not actual gameplay outcomes. A deck that scores well on synergy might still lose to specific metagame threats.

Future Work

- **Function approximation**: Replace the Q-table with a neural network to generalize across similar cards and states.
- **Prioritized sweeping**: Replace random planning sampling with prioritized sweeping (Moore & Atkeson, 1993) to focus planning on recently-changed transitions.
- **Multi-objective reward**: Separate reward signals for synergy, power level, and budget could give the agent more nuanced optimization targets.
- **Transfer learning**: Pre-train on a large public card pool, then fine-tune on the personal collection.

References

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4), 160-163.

- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292.
- Towers, M., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*.